



Operations Research

Publication details, including instructions for authors and subscription information:
<http://pubsonline.informs.org>

Technical Note—Shortest-Path Algorithms: A Comparison

Bruce Golden,

To cite this article:

Bruce Golden, (1976) Technical Note—Shortest-Path Algorithms: A Comparison. *Operations Research* 24(6):1164-1168.
<https://doi.org/10.1287/opre.24.6.1164>

Full terms and conditions of use: <https://pubsonline.informs.org/Publications/Librarians-Portal/PubsOnLine-Terms-and-Conditions>

This article may be used only for the purposes of research, teaching, and/or private study. Commercial use or systematic downloading (by robots or other automatic processes) is prohibited without explicit Publisher approval, unless otherwise noted. For more information, contact permissions@informs.org.

The Publisher does not warrant or guarantee the article's accuracy, completeness, merchantability, fitness for a particular purpose, or non-infringement. Descriptions of, or references to, products or publications, or inclusion of an advertisement in this article, neither constitutes nor implies a guarantee, endorsement, or support of claims made of that product, publication, or service.

© 1976 INFORMS

Please scroll down for article—it is on subsequent pages



With 12,500 members from nearly 90 countries, INFORMS is the largest international association of operations research (O.R.) and analytics professionals and students. INFORMS provides unique networking and learning opportunities for individual professionals, and organizations of all types and sizes, to better understand and use O.R. and analytics tools and methods to transform strategic visions and achieve better outcomes.

For more information on INFORMS, its publications, membership, or meetings visit <http://www.informs.org>

Shortest-Path Algorithms: A Comparison

BRUCE GOLDEN

Massachusetts Institute of Technology, Cambridge, Massachusetts

(Received original May 5, 1975; final, October 8, 1975)

In this note we present some computational evidence to suggest that a version of Bellman's shortest-path algorithm outperforms Treesort-Dijkstra's for a certain class of networks.

MANY APPLICATIONS dealing with transportation and communication networks require the calculation of shortest paths. We discuss here the specific problem of finding the shortest paths from one node to all others. Our objective is to present some computational experience to support the claim that a version of Bellman's algorithm outperforms Treesort-Dijkstra's for a certain class of networks. This note complements a recent paper by Pape [8] and indicates that, for the class of networks under consideration, Pape's reluctance to use a variable length list of nodes to be scanned is unwarranted. (The phrase "variable length list" refers to the fact that no realistic a priori bound on the length of the list is known.)

A sequence of distinct arcs $(a_1, a_2, \dots, a_p)$, where a_t and a_{t+1} are adjacent for $t = 1, \dots, p - 1$ is called a path; a route is defined as a sequence of adjacent arcs that need not be distinct. We seek the shortest paths from the origin to all other nodes. Dreyfus [3] discusses several such algorithms in his survey paper, primarily from the viewpoint of computational complexity.

Dijkstra's algorithm (reference 2) requires on the order of NN^2 additions and NN^2 comparisons in the worst case where NN is the number of nodes. This algorithm is a "label-setting" method that assigns permanent labels as it proceeds. Initially, the set T consists of the origin alone. T is augmented one node at a time so that at each step T is a set of permanently labeled nodes that corresponds to the shortest-path tree for all nodes in T . Termination occurs when all nodes of the graph are in T . Labeling methods for computing shortest paths can be divided into two general classes, "label-setting" and "label-correcting" (see reference 5). Label-setting methods are valid only for non-negative arc lengths.

Bellman's algorithm (reference 1) solves the problem in at most NN^3 additions and comparisons or detects the existence of a negative cycle. This algorithm is an example of the label-correcting approach, in which no

node labels are considered permanent until, when at termination, they all are.

If negative cycles exist, then clearly there can be no shortest route on a network. The shortest-path problem in that case has been shown to be combinatorially equivalent to the traveling salesman problem [4].

The performance of shortest-path algorithms is heavily dependent upon the following three factors: (i) the sparseness of the network, (ii) list processing and network representation in the computer code, and (iii) distance measures on the arcs.

The topological structure of the network clearly exerts a major influence on running time for any graph algorithm. Theoretical upper bounds have been calculated assuming a complete graph with every pair of nodes connected by an arc. If the graph is sparse, running times may be reduced significantly [5]. (The same observation has been exploited with the minimum spanning tree problem [7].) Computer representation of networks is discussed in reference 5. Distance measures will be mentioned later.

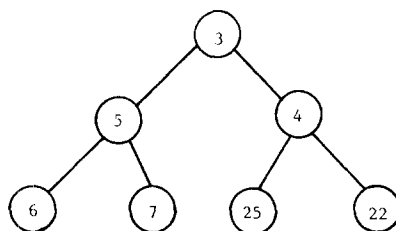


Figure 1

1. IMPLEMENTATION

For sparse graphs Bellman's algorithm can be made quite efficient using a list structure that keeps track of which nodes can potentially label other nodes. This list is of nodes to be scanned.

The origin is the first element on the list. Those nodes that can be reached directly from the origin are labeled and placed on the list. We proceed downward from the top of the list and scan each member of the list, possibly adding new members if a new label is less than a current label. When we have scanned the entire list, we have the shortest-path tree. At the same time we trace the shortest-path tree through the predecessor labels for each node. A flag associated with the active (unscanned) members of the list prevents us from placing one node on the active list more than once at any time.

In the Dijkstra algorithm a primary computational concern involves the determination of the minimum distance node at each step. We have implemented a modified Dijkstra algorithm, where Floyd's treesort algorithm

is used for the sorting of these distances. This approach has been studied by Johnson [6] and by Kershenbaum and Van Slyke [7]. The node distances $d_1, d_2, \dots, d_m$, where $m = 2^k - 1$, are arranged in a binary tree

TABLE I
COMPUTATIONAL EXPERIENCE: BELLMAN VS. DIJKSTRA (SERIES A) (AVERAGE
RUNNING TIMES GIVEN IN SECONDS)

NN	R	DIJKSTRA TIME	BELLMAN TIME	AVERAGE LIST LENGTH	MAXIMUM LIST LENGTH
50	3	.021	.014	60.0	72
	4	.024	.013	55.6	66
	5	.027	.017	58.5	74
	6	.029	.021	59.7	70
100	3	.047	.026	117.6	131
	4	.055	.032	128.2	139
	5	.062	.037	124.6	141
	6	.065	.044	125.7	149
150	3	.084	.040	176.1	185
	4	.087	.053	197.6	235
	5	.102	.060	201.3	232
	6	.103	.066	195.1	220
250	3	.148	.072	323.3	389
	4	.164	.090	338.9	386
	5	.178	.103	327.1	363
	6	.190	.116	333.0	377
350	3	.230	.108	466.6	518
	4	.251	.128	473.8	523
	5	.264	.152	501.3	606
	6	.277	.171	509.2	595
500	3	.327	.155	703.0	819
	4	.368	.177	658.5	766
	5	.406	.214	699.7	802
	6	.422	.249	722.5	782
750	3	.538	.229	1012.6	1136
	4	.573	.286	1090.7	1262
	5	.637	.334	1068.2	1140
	6	.690	.402	1110.5	1241
1000	3	.733	.309	1389.5	1589
	4	.809	.383	1447.6	1661
	5	.848	.433	1462.0	1629
	6	.894	.518	1535.8	1676

with k levels, called a heap. The essential property of a heap is that $d_i \leq d_{2i}$ and $d_i \leq d_{2i+1}$. Figure 1 shows a heap for $k = 3$ ($d_1 = 3, d_2 = 5, d_3 = 4$, and so on).

If the list length is $l < 2^k - 1$, we can fill positions $l + 1, \dots, 2^k - 1$ for the smallest k such that $l < 2^k - 1$ with distances of ∞ . Clearly, d_1 is the minimum node distance under consideration. If we remove d_1 from the heap, a new heap can be constructed with relative ease. The modified Dijkstra algorithm has node distances d_i composing the heap for all nodes

i that are not yet in T . After a new node has been added to T and has been scanned, we remove the top node of the heap and form a new heap.

2. COMPUTATIONAL EXPERIENCE

Bellman's and Dijkstra's algorithms have been coded and tested on M.I.T.'s IBM 370/168 system assuming non-negative arc distances. Two groups of networks are studied: Series A and Series B. Node coordinates are generated from a uniform probability distribution over a rectangular grid, and then the euclidean distances are calculated between "randomly selected" pairs. These pairs are chosen in such a way that the out-degree of every node is equal to R for Series A, and the out-degree of each node

TABLE II
COMPUTATIONAL EXPERIENCE: BELLMAN VS. DIJKSTRA (SERIES B) (AVERAGE
RUNNING TIMES GIVEN IN SECONDS)

NN	DIJKSTRA TIME	BELLMAN TIME	AVERAGE LIST LENGTH	MAXIMUM LIST LENGTH
50	.028	.015	58.7	64
100	.055	.030	117.2	132
150	.091	.052	193.3	211
250	.165	.088	323.6	358
350	.238	.133	497.5	574
500	.365	.184	695.9	886
750	.574	.275	1050.9	1305
1000	.808	.382	1422.7	1501

takes on the value 2, 3, 4, 5, or 6 with equal probability for Series B. A computational consideration in Bellman's algorithm is how long the list of nodes to be scanned grows. With euclidean distances and sparse networks, one would expect that most nodes are not put on the list to be scanned more than once (precisely because of the triangle inequality).

For Series A we generated ten networks of NN nodes, where each node had fixed out-degree R for $NN = 50, 100, 150, 250, 350, 750, 1000$ and $R = 3, 4, 5, 6$. For Series B we generated and tested ten networks of NN nodes, where the out-degree for each node varied from 2 to 6. We then applied Bellman's and Dijkstra's algorithms to determine shortest paths. The mean running times for Series A and Series B are shown in Table I and Table II, respectively. In addition, the average length of the list of nodes to be scanned and the maximum length are displayed.

For a given R (Series A) our results indicate that the relationship be-

tween NN and running time is nearly linear for both algorithms. The Series B results are similar. Bellman's algorithm clearly outperforms Dijkstra's algorithm; running times from Dijkstra's algorithm are about twice the running times from Bellman's. Interestingly, as the number of nodes increases, Bellman's algorithm becomes more and more attractive relative to Dijkstra's.

Our computational experience suggests that the variable length list does not become a great deal longer than NN . In fact, for $NN \leq 1000$, we can be confident that the list length will not exceed $2 \cdot NN$ for the class of networks discussed in this paper. Key properties in this class include network sparsity and euclidean distances.

ACKNOWLEDGMENTS

The author would like to thank Aaron Kershenbaum of Network Analysis Corporation and Professor Gabriel Handler of M.I.T. for the opportunity to perform this computational comparison. Helpful comments by Professor Thomas Magnanti of M.I.T. are appreciated. In addition, the author is grateful to NAC for the use of their modified Dijkstra code.

REFERENCES

1. R. BELLMAN, "On a Routing Problem," *Quart. Appl. Math.* **16**, 87-90 (1958).
2. E. DIJKSTRA, "A Note on Two Problems in Connection with Graphs," *Numerische Mathematik* **1**, 269-271 (1959).
3. S. DREYFUS, "An Appraisal of Some Shortest-Path Algorithms," *Opns. Res.* **17**, 395-412 (1969).
4. L. FORD, JR., AND D. R. FULKERSON, *Flows in Networks*, Princeton University Press, Princeton, N.J., 1962.
5. J. GILSINN AND C. WITZGALL, "A Performance Comparison of Labelling Algorithms for Calculating Shortest Path Trees," National Bureau of Standards, Technical Note 772, May 1973.
6. E. JOHNSON, "On Shortest Paths and Sorting," pp. 510-517, in *Proceedings of 1972 ACM Conference*, Boston, August 1972.
7. A. KERSHENBAUM AND R. VAN SLYKE, "Computing Minimum Spanning Trees Efficiently," pp. 518-529, in *Proceedings of 1972 ACM Conference*, Boston, August, 1972.
8. U. PAPE, "Implementation and Efficiency of Moore-Algorithms for the Shortest Route Problem," *Math. Programming* **7**, 212-222 (1974).

Copyright 1976, by INFORMS, all rights reserved. Copyright of Operations Research is the property of INFORMS: Institute for Operations Research and its content may not be copied or emailed to multiple sites or posted to a listserv without the copyright holder's express written permission. However, users may print, download, or email articles for individual use.